

Can Coding Agents Reproduce Findings in Computational Materials Science?

Ziyang Huang, Yi Cao*, Ali K. Shargh*, Jing Luo, Ruidong Mei
 Mohd Zaki, Zhan Liu, Wyatt Bunstine, William Jurayj, Somdatta Goswami
 Tyrel McQueen, Michael Shields, Jaafar El-Awady, Paulette Clancy
 Benjamin Van Durme, Nicholas Andrews[♡], William Walden[♡], Daniel Khashabi[♡]

Johns Hopkins University
 {zhuang86, noa, wwalden1, danielk}@jhu.edu

	Homepage	https://jhu-clsp.github.io/AutoMat
	Source Code	https://github.com/JHU-CLSP/AutoMat
	HF Dataset	https://hf.co/datasets/jhu-clsp/AutoMat

Abstract

Large language models are increasingly deployed as autonomous coding agents and have achieved remarkably strong performance on software engineering benchmarks. However, it is unclear whether such success transfers to computational scientific workflows, where tasks require not only strong coding ability, but also the ability to navigate complex, domain-specific procedures and to interpret results in the context of scientific claims. To address this question, we present AUTOMAT, a benchmark for evaluating LLM-based agents’ ability to reproduce claims from computational materials science. AUTOMAT poses three interrelated challenges: recovering underspecified computational procedures, navigating specialized toolchains, and determining whether the resulting evidence supports a claim. By working closely with subject matter experts, we curate a set of claims from real materials science papers to test whether coding agents can recover and execute the end-to-end workflow needed to support (or undermine) such claims. We then evaluate multiple representative coding agent settings across several foundation models. Our results show that current LLM-based agents obtain low overall success rates on AUTOMAT, with the best-performing setting achieving a success rate of only 54.1%. Error analysis further reveals that agents perform worst when workflows must be reconstructed from paper text alone and that they fail primarily due to incomplete procedures, methodological deviations, and execution fragility. Taken together, these findings position AUTOMAT as both a benchmark for computational scientific reproducibility and a tool for diagnosing the current limitations of agentic systems in AI-for-science settings.¹

1 Introduction

Recent advances in large language models (LLMs) have driven rapid progress in code generation and agentic coding assistants, leading to a wave of agents that can write, run, and debug code in interactive environments (Chen et al., 2021; Yang et al., 2024). These systems have attracted substantial attention because they perform well on software engineering-style tasks, where requirements are explicit and where correctness is validated via tests,

*Equal contribution. [♡]Equal advising.

¹Large language models were used as part of the evaluation pipeline in the form of an LLM-based judge, as described in §4.3. Aside from this methodological use, the authors used LLM-based tools only for editorial and implementation assistance. The authors assume full responsibility for all content.

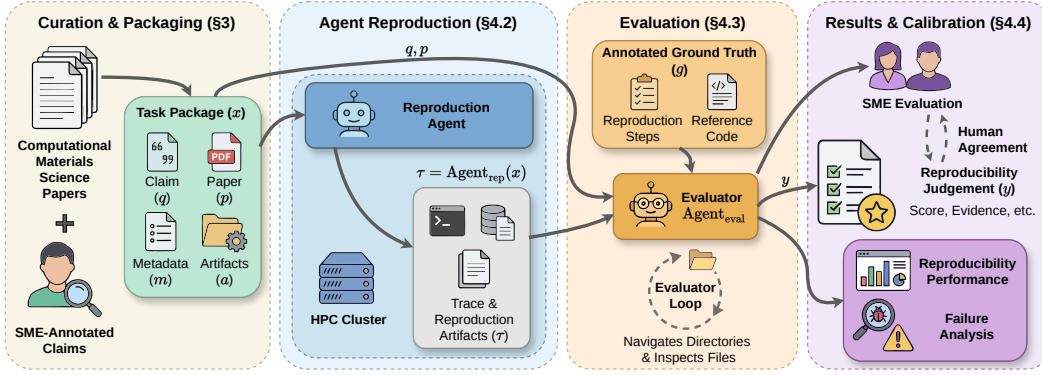


Figure 1: Overview of AUTOMAT. Claims from domain experts are packaged into runnable tasks and executed by reproduction agents in an HPC environment. A separate evaluator agent then inspects the resulting trace and artifacts to assign a reproducibility judgment.

static checks (Jimenez et al., 2024), as well as other intermediate feedback (e.g., linting and compilation). An increasingly mature ecosystem of benchmarks reflects this focus, evaluating how well LLMs generate, modify, and execute code in realistic development settings (Jimenez et al., 2024; Shah et al., 2024; Jain et al., 2025).

However, strong performance on *software engineering tasks* does not necessarily imply strong performance in *reproducing scientific findings* (Xiang et al., 2025; Hu et al., 2025; Siegel et al., 2024). Reproducing a scientific claim requires much more than merely implementing a well-specified function; agents must recover an end-to-end workflow from partial information, navigate domain-specific toolchains, and produce evidence that supports (or undermines) a paper-level result (Krafczyk et al., 2021). This makes scientific reproduction a distinct regime—underspecified, tool-rich, and failure-prone—where reasonable-looking outputs can still be scientifically wrong (Antunes & Hill, 2024; Hassan et al., 2025).

Computational materials science is a particularly stringent domain for this evaluation, since typical claims rely on complex multi-stage pipelines that combine simulation, feature extraction, model training, and post-processing, and often involve specialized external dependencies and tacit methodological choices (Persaud et al., 2024; Li et al., 2024). Even when code is released, reproducing a main result may require matching subtle, implicit choices that are rarely documented yet often essential in practice (Lejaeghere et al., 2016). This makes computational materials science a natural stress test for whether coding agents can function as reliable scientific assistants rather than merely as capable programmers.

In this work, we introduce AUTOMAT, a benchmark that operationalizes this challenge for LLM-based agents. AUTOMAT comprises 85 carefully curated and annotated computational materials science claims curated by subject matter experts (SMEs) from recent publications across diverse subdomains. Each instance is framed as an unsupervised reproduction task (Figure 1): given a claim paired with released artifacts (when available), an agent should identify the required computational procedure, implement or adapt the necessary code, configure the environment, execute the workflow, and produce evidence supporting or undermining the claim. To preserve realism while maintaining reliable evaluation, we combine a standardized task format, manual curation, and artifact-grounded assessment, so that the benchmark assesses reproduction *per se* rather than idiosyncrasies of task construction.

We then use AUTOMAT to evaluate widely used coding agents in this scientific setting. We compare several off-the-shelf agentic systems as well as a customized, orchestrated workflow across multiple foundation models and configurations. In summary:

- We introduce AUTOMAT, a benchmark of 85 SME-curated computational materials science claims for evaluating end-to-end scientific reproducibility with LLM agents.
- We formulate claim-level scientific reproduction as a runnable benchmark task, with standardized task packaging and artifact-grounded, SME-calibrated assessment.

- We present an empirical study of five representative coding agent settings, showing low overall success, a large gap between from-paper and from-artifact tasks, and recurrent failures in agents’ procedure recovery and workflow execution.

2 Related work

Reproducibility Benchmarks Recent benchmarks have begun to evaluate LLM agents on computational reproducibility, though under different task definitions and domains. CORE-Bench (Siegel et al., 2024) studies whether agents can reproduce study results when given code, data, and task materials, while REPRO-Bench (Hu et al., 2025) focuses on *end-to-end* reproducibility in social science, where agents receive a paper, a reproduction package, and major findings, and judge whether reproduced results support those findings. SciReplicate-Bench (Xiang et al., 2025) instead studies algorithmic reproduction from NLP papers, asking agents to generate code from paper descriptions and evaluating them with execution-based tests and code quality metrics. PaperBench (Starace et al., 2025) similarly targets replication of AI research papers but requires agents to start from scratch. AUTOMAT is inspired by these efforts but differs in both domain and task definition: it targets *unsupervised* reproduction of *computational materials science claims*, where success is assessed at the claim level and requires end-to-end reproduction in a domain with highly specialized conventions and toolchains. Table 1 summarizes differences between AUTOMAT and key prior works.

Benchmark	Main task	Agent input	Task unit	Domain
AUTOMAT (our work)	Reproduce a scientific claim: reconstruct and execute the workflow needed to produce evidence for a specific paper claim.	Claim, paper, metadata, optional artifacts	Claim-level	Computational materials science
REPRO-Bench (Hu et al., 2025)	Assess whether results reproduced from the paper’s reproduction package are consistent with selected major findings .	Paper, reproduction package, major findings	Paper-level, via selected findings/results	Social science
CORE-Bench (Siegel et al., 2024)	Use provided code and data to regenerate specified results.	Code, data, task materials	Task-level	Computer science, social science, medicine

Table 1: Comparison between AUTOMAT and related reproducibility benchmarks. We distinguish benchmarks by the object of evaluation: AUTOMAT targets claim-level scientific reproduction, REPRO-Bench targets paper-level major findings, and CORE-Bench targets predefined study results from provided materials.

Computational Materials Science Benchmarks for LLMs A growing body of work evaluates LLMs in materials science, but most of this work focuses on basic knowledge, question answering, feasibility prediction, or tool use in isolation rather than full, multi-step reproduction of workflows. Matter-of-Fact (Jansen et al., 2025), for example, frames materials science progress as a feasibility/claim verification problem, while MatTools (Liu et al., 2025) and SciCode (Tian et al., 2024) evaluate materials-related tool use and scientific coding. AUTOMAT is complementary to these benchmarks: instead of asking whether a claim is feasible or whether a model can use a tool correctly, AUTOMAT asks whether an agent can reconstruct, execute, and validate a computational materials science claim end-to-end.

Coding-Agent Interfaces AUTOMAT evaluates a class of systems that are increasingly deployed as general-purpose coding copilots. Terminal-based agents such as Claude Code², Codex CLI³, OpenHands CLI⁴ (Wang et al., 2025), and Kimi Code⁵ support file editing,

²Claude Code (Anthropic): <https://code.claude.com/docs/en/overview>

³Codex CLI (OpenAI): <https://github.com/openai/codex>

⁴OpenHands CLI (OpenHands): <https://github.com/OpenHands/OpenHands-CLI>

⁵Kimi Code (MoonshotAI): <https://github.com/MoonshotAI/kimi-cli>

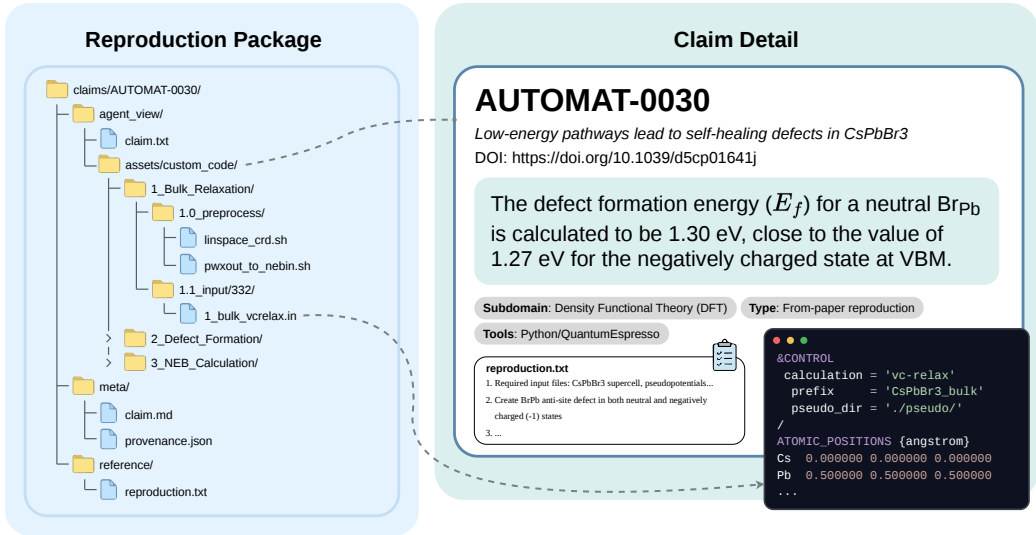


Figure 2: An example claim reproduction package from AUTOMAT.

shell execution, and iterative debugging in local workspaces and are typically positioned as software engineering assistants that work independently on a target task. We treat these systems as representative interfaces and ask whether their strength in software engineering transfers to scientific reproduction, where success depends on workflow recovery, long-horizon execution, and grounded validation—rather than passing tests alone.

3 AUTOMAT benchmark

Here, we describe how AUTOMAT is constructed, how claims are packaged into runnable tasks, and what types of reproducibility problems the benchmark contains.

3.1 Overview

Figure 1 summarizes AUTOMAT’s construction. We begin with a curated set of computational materials science papers and SME-annotated claims. Each claim is packaged into a runnable task with associated metadata and available artifacts, executed in a resource-controlled environment representative of modern HPC workflows, and then evaluated with a separate artifact-grounded LLM judge.

3.2 Benchmark curation

Figure 2 shows an example task package. AUTOMAT contains three broad claim types that differ in the amount of procedural recovery required from the agent: *from-paper reproduction*, *from-artifact reproduction*, and *from-artifact interpretation*, each of which introduces unique challenges. We discuss these claim types in detail in §3.3.

Claim collection and annotation From a collection of computational materials science papers, SMEs—senior Ph.D. students and postdoctoral researchers who have published at least one peer-reviewed materials science journal paper as a first author—identify claims that are suitable for reproducibility assessment and annotate them with supporting information such as paper provenance, expected outcome, and intended reproduction steps. These claims span several major computational materials science subdomains, including Statistical/Machine Learning methods, Density Functional Theory, Molecular Dynamics, and Discrete Dislocation Dynamics, each of which introduces distinct reproducibility challenges.

Appendix E includes short, SME-authored notes that offer further context on these domains and their reproducibility challenges.

We represent each AUTOMAT example x as

$$x = (q, p, m, a),$$

where q is the claim text, p is the source paper, m is the metadata file, and a denotes any accompanying artifacts such as scripts, custom codebases, data, or other relevant materials. SMEs are instructed to write claims centered on numerical results that serve as critical evidence for the paper’s main conclusions. This emphasis distinguishes AUTOMAT from benchmarks that focus on broad qualitative statements available in natural language critiques (Ou et al., 2025): our goal is to evaluate whether agents can recover and validate *quantitatively checkable* scientific claims.

Task packaging After collection, each curated claim is converted into a self-contained task package that exposes the claim, paper⁶, metadata file, and any provided artifacts, while concealing provenance and expert annotations for later evaluation. We treat artifacts as optional because reproducibility requirements vary across claims. Some tasks can be reproduced using standard software and configuration inputs described in the source paper, whereas others depend on bespoke codebases, packaged datasets, or specialized scripts. For every claim, we additionally collect a SME-authored, step-by-step reproduction procedure, which is reserved for evaluation and remains hidden from the agent.

3.3 Claim types and benchmark statistics

AUTOMAT contains 85 claims spanning three reproduction types. *From-paper reproduction* includes claims that are reproducible with generic tools and public resources described in their source paper, or paired with artifacts, such as templates or examples. *From-artifact reproduction* includes claims paired with artifacts consisting of custom codebases, pretrained models, or datasets that are essential to the reproduction path. Finally, *from-artifact interpretation* includes claims with final outputs from simulation workflows; reproduction consists primarily of post-hoc analysis and interpretation.

Claim Category Distribution by Reproduction Type

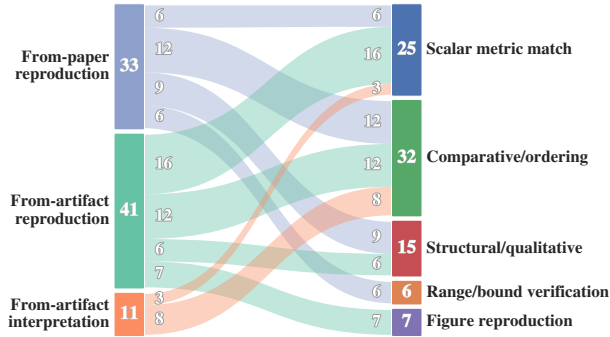


Figure 3: AUTOMAT claim distribution by reproduction type (left) and verification category (right).

We further categorize each claim based on the type of verification required: *scalar metric match*, *comparative/ordering*, *structural/qualitative*, *range/bound verification*, *figure reproduction*. Full taxonomy definitions are provided in Appendix A.1. Figure 3 shows the joint distribution of claims across reproduction types and verification categories.

4 Experiment and evaluation setup

Below, we detail the five agent settings that we evaluate, the protocol used to produce a reproduction attempt for each claim, and the artifact-grounded evaluation procedure used to score the reproduction runs. We also describe how the LLM-based evaluator is calibrated against SME judgments.

⁶While we do not observe agents searching for implementation or accessing release artifacts through links in the paper, we remove URLs that might lead to leakage before launching agents.

4.1 Agent settings

We evaluate five agent settings on AUTOMAT. One is an AUTOMAT task-specific orchestrated agent (**Orch.**) built to follow a structured reproduction workflow using the Claude Agent SDK with Claude Sonnet 4.6 (Anthropic, 2026b). The other four are general-purpose coding agents: Claude Code (CC) with (1) Claude Opus 4.6 (Anthropic, 2026a), (2) Claude Sonnet 4.6, and (3) Kimi K2.5 (Kimi; Kimi Team et al., 2026); and (4) OpenAI’s Codex CLI (**Codex**) with GPT-5.4 (OpenAI, 2026).

This setup serves two purposes. First, it provides a cross-system view of current end-to-end reproducibility performance. Second, it allows a comparison between the benchmark-specific orchestrated agent and widely used off-the-shelf coding agents (Claude Code, Codex), thus enabling us to study the effect of workflow design more directly. Because our experimental budget is limited, these settings should be interpreted as representative end-to-end systems rather than as an exhaustive ablation study across models and interfaces. Full model identifiers, interface versions, and implementation details are provided in Appendix B. Runtime environment and hardware configuration are detailed in Appendix C.

4.2 Reproduction attempt

Given a task instance x , consisting of a claim q , paper p , metadata m , and optional artifacts a , AUTOMAT launches an LLM-based agent to investigate whether the target claim can be reproduced. The agent reads the paper and metadata, plans an investigation, executes commands, inspects outputs, and writes a final report. The run takes place in a resource-controlled execution setting representative of modern HPC workflows, such as environments managed by Slurm (Yoo et al., 2003) and, when appropriate, containerized software stacks such as Singularity (Kurtzer et al., 2017). We denote the resulting trajectory as $\tau = \text{Agent}_{\text{rep}}(x)$, where τ includes the full execution trace, generated files, terminal logs, and the agent’s final analysis. This setup captures end-to-end autonomous investigation rather than text-only reasoning.

Task-specific orchestrated agent. Among the five evaluated settings, one is a benchmark-specific orchestrated agent (**Orch.**) designed to impose more structure on the reproduction process. It separates a run into four phases: read-only planning, environment preparation, deterministic execution with LLM-based failure diagnosis, and result extraction with self-assessment. The goal of this design is not to introduce additional scientific knowledge, but to improve auditability and process control compared with more free-form CLI-based agents. We use this system primarily to support the controlled comparison in §5.2. Implementation details are provided in Appendix B.4.

4.3 Artifact collection and evaluation

After execution, AUTOMAT aggregates the complete evidence produced during the run, including logs, commands, intermediate outputs, reproduction artifacts, and the agent’s written report. These materials form the basis for reproducibility assessment. By preserving the full trajectory rather than only the final verdict, AUTOMAT enables investigation both of *whether* the agent succeeded and of *how* it carried out its attempt.

A separate LLM-based evaluator agent then judges the reproduction attempt. Unlike a fixed-prompt judge that only consumes a static prompt, the evaluator is allowed to navigate the produced artifact directory, inspect files, and use tools to gather the information needed for assessment. The evaluator receives the SME-annotated ground-truth reproduction steps, g , including reference code (when available), along with the original claim q , the source paper p , and the execution trace τ with produced artifacts. Formally, we write

$$y = \text{Agent}_{\text{eval}}(g, q, p, \tau),$$

where y is a structured evaluation containing an overall score, dimension-level assessments, and justifications.

The evaluator assigns an overall score on a five-point scale: 5 (*fully reproduced*), 4 (*mostly reproduced*), 3 (*partially reproduced*), 2 (*mostly failed*), 1 (*failed*). We define *successful* reproductions as those that receive an overall score of at least 4. The overall score is supported by evidence along five dimensions: methodological fidelity, execution competence, result accuracy, completeness, and scientific rigor. For each dimension, the evaluator similarly provides a 5-point Likert-scale score and justification, and it may introduce claim-specific sub-dimensions to capture domain-specific criteria. The evaluator also summarizes what the agent did correctly and highlights key errors that prevented a stronger reproduction, explicitly comparing its assessment to the agent’s self-judgment when available. All evaluation outputs are written to the artifact directory.

4.4 Judge calibration against human SMEs

To assess whether automated evaluation is sufficiently aligned with expert judgment for benchmark-scale use, we calibrate the evaluator against blinded assessment from qualified SMEs on a sampled subset ($n = 40$) of AUTOMAT runs. We construct this subset using stratified sampling to cover different agent systems and claim types. For each selected run, SMEs are given the same task materials, execution traces, and produced artifacts used for evaluation. To reduce anchoring effects, SMEs do not see the agent’s final answer, self-evaluation, or the LLM judge’s assessment. Instead, they score each run independently using the same five-point scale and the same five evaluation dimensions. On this calibration subset, the evaluator achieves a quadratic-weighted kappa (Cohen, 1968) of 0.69, suggesting substantial agreement, and a within-1 difference accuracy of 0.80—both indicating that the evaluator’s judgments are sufficiently aligned with SME assessments to support large-scale analysis.

5 Findings and analysis

We study LLM agents for scientific reproducibility along three axes: overall capability, the role of agent design, and the nature of their failures. Concretely, we aim to answer the following research questions.

- **RQ1:** how well can current LLM-based coding agents reproduce computational materials science claims end-to-end?
- **RQ2:** does task-specific orchestration improve reproduction performance over general-purpose coding agents?
- **RQ3:** what failure modes tend to prevent successful reproductions on AUTOMAT?

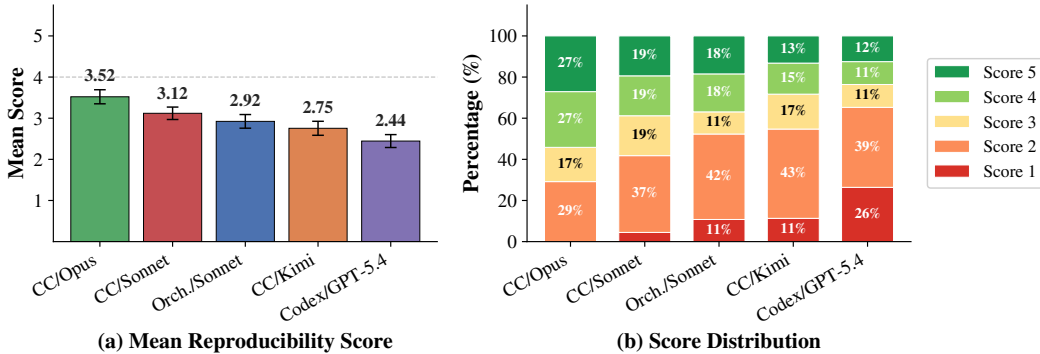


Figure 4: Overall reproducibility performance on AUTOMAT across five agent settings. (a) Mean overall reproducibility score assigned by the evaluator on a 1–5 scale. (b) Distribution of evaluator-assigned overall scores across all benchmark instances.

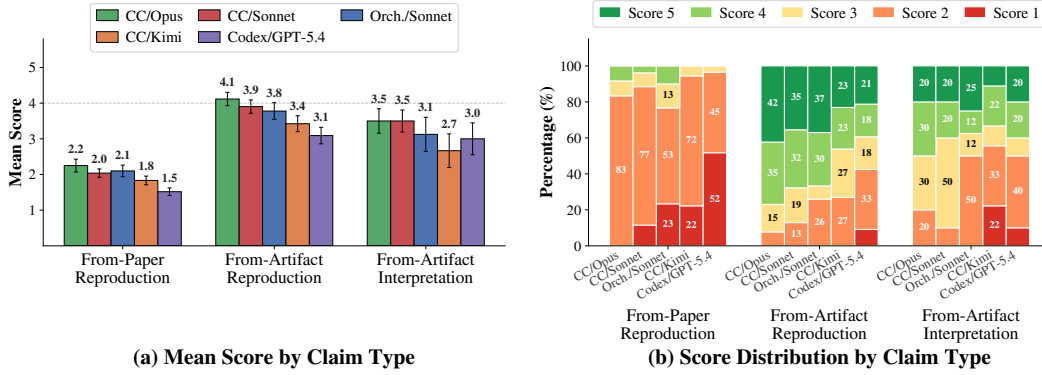


Figure 5: (a) Mean overall reproducibility score for each agent setting, broken down by claim type: *from-paper reproduction*, *from-artifact reproduction*, and *from-artifact interpretation*. Error bars denote standard errors. Higher scores indicate better performance. (b) Score distribution within each claim type.

5.1 RQ1: How well can current LLM-based coding agents reproduce computational materials science claims end-to-end?

Current LLM agents struggle on AUTOMAT. Figure 4(a) shows that performance remains limited across all systems. Claude Code with Opus is the strongest configuration we evaluate, with a mean overall reproducibility score of 3.52 and a success rate of 54.1%, while Codex with GPT-5.4 is the weakest overall, achieving a mean score of 2.44 and a success rate of 23.5%. Since scores of 4 and 5 correspond to strong reproductions, this means that *none* of the tested systems is yet a reliable autonomous assistant for scientific reproducibility in this setting, and even the best-performing system succeeds on only about half of all instances. The score distributions in Figure 4(b) reinforce this picture: across systems, a large fraction of runs receive only mid-range scores, indicating that current agents often make partial progress without reaching a complete and convincing reproduction.

Performance varies sharply by claim type. Figure 5 shows detailed score distributions by claim type. Both the mean score and success rate confirm that from-paper reproduction is by far the hardest setting, with mean scores ranging from 1.5 to 2.2 and *near-zero* success rates across all systems. This suggests that recovering an underspecified workflow from paper text alone remains a major bottleneck. By contrast, from-artifact reproduction is substantially easier, with mean scores between 3.1 and 4.1 and success rates between 39% and 77%, indicating that access to executable artifacts removes much of the ambiguity in the task and shifts the challenges from full workflow recovery to execution and adaptation.

The from-artifact interpretation setting reveals a complementary limitation. Even when outputs are already available, performance remains uneven, with success rates ranging from 33% to 50% across the systems. This indicates that scientific reproducibility is not only an execution problem but also an interpretation problem: agents should determine whether existing outputs actually support the target claim and justify that judgment convincingly, which remains difficult despite artifacts being available.

5.2 RQ2: Does task-specific orchestration improve performance over general-purpose coding agents?

Task-specific orchestration does not improve overall reproducibility in a controlled comparison. Figure 6 compares our orchestrated agent against Claude Code, both running on Claude Sonnet 4.6. The paired per-claim win/tie/loss breakdown is dominated by ties (45.0%), with wins and losses remaining relatively balanced. This indicates that the orchestrated workflow does not yield a clear overall advantage in end-to-end success.

At the dimension level, the orchestrated agent scores statistically higher ($p < 0.05$) only in scientific rigor. This suggests that task-specific orchestration improves the quality of scientific reasoning and evidence handling more than it improves reproduction success *per se*. Meanwhile, the orchestrated agent *underperforms* Claude Code with Sonnet on several other evaluation dimensions, suggesting that task-specific orchestration introduces trade-offs rather than uniformly improving performance. Since Claude Code supports a more fluid interaction loop than the orchestrated agent (which has a phased design), the latter is at a disadvantage in tasks that depend on opportunistic repair and incremental adaptation over long-horizon workflows. By contrast, the orchestrated design appears to favor caution and evidence-grounded analysis, which is consistent with its higher scientific rigor scores.

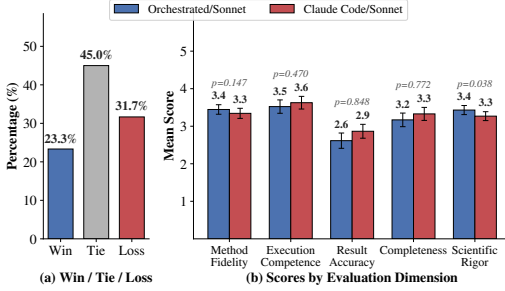


Figure 6: Controlled (same-backbone) comparison between the orchestrated agent and Claude Code with Sonnet 4.6. (a) Per-claim win/tie/loss breakdown based on the evaluator’s overall reproducibility score. (b) Mean scores by evaluation dimension, with one-sided Wilcoxon signed-rank test p -values shown for each dimension.

	Proc. Incompl.	Method. Deviation	Resource & Exec. Failure	Overconf. Assessment	Circular Reasoning	Under-train / Simplif.
Claude Code Opus	40 47.1%	16 18.8%	12 14.1%	8 9.4%	6 7.1%	2 2.4%
Claude Code Sonnet	44 51.8%	26 30.6%	25 29.4%	17 20.0%	8 9.4%	10 11.8%
Orchestrated Sonnet	50 58.8%	25 29.4%	34 40.0%	9 10.6%	4 4.7%	8 9.4%
Claude Code Kimi	35 41.2%	27 31.8%	22 25.9%	18 21.2%	19 22.4%	4 4.7%
Codex GPT-5.4	56 65.9%	17 20.0%	14 16.5%	6 7.1%	5 5.9%	4 4.7%

Figure 7: Failure-mode heatmap across agent settings on AUTOMAT. Each cell shows the count and percentage of all runs in that setting assigned the corresponding failure label. A single run may receive multiple failure labels. See Appendix A.2 for full definitions of the failure taxonomy.

5.3 RQ3: What failure modes tend to prevent successful reproductions on AUTOMAT?

Failures on AUTOMAT are dominated by breakdowns in carrying out the intended scientific procedure, rather than by simple downstream mismatches in outputs, as shown in Figure 7. Across systems, and among all runs in each setting, the most common failure modes are *procedural incompleteness* and *methodological deviation*, indicating that agents often either fail to complete the necessary workflow or pursue a reproduction path that diverges from the methodology that the claim demands. *Resource and execution failures* are also common, especially for orchestrated agents, showing that long-horizon scientific workflows remain fragile in practice. By contrast, *circular reasoning*, *overconfident assessment*, and *under-training / simplification* occur less frequently overall.

The heatmap also reveals distinct behaviors across systems. For example, failures of Codex with GPT-5.4 are particularly driven by procedural incompleteness, suggesting that many failures arise from incomplete workflow execution rather than from an inability to reason about the task. The orchestrated agent exhibits fewer instances of overconfidence, but still suffers from incomplete procedures and execution breakdowns, consistent with its stronger scientific rigor but limited overall success. These patterns broadly help explain the results in §5.1, in which agents often make partial progress but fail to recover a complete reproduction.

To make these failure categories concrete, we include three representative case studies. Appendix F.1 illustrates procedural incompleteness: the agent correctly identifies the scientific gap but fails to translate that diagnosis into a workable fallback reproduction strategy. Appendix F.2 shows how methodological deviation at the dependency-handling stage can cause a complete task failure. Appendix F.3 documents a subtler failure where the agent reproduces the reported scalar metric almost exactly but does so on a deviated evaluation subset, recovering the number without verifying the underlying scientific conclusion.

6 Conclusion

We present AUTOMAT, a benchmark for evaluating whether LLM-based coding agents can reproduce computational materials science claims end-to-end. Across 85 expert-curated claims, our results show that current coding agents fall well short of reliable scientific reproducibility, especially when agents must reconstruct underspecified workflows from the text of papers alone. Error analysis further shows that the dominant bottlenecks are procedural incompleteness, methodological deviation, and execution fragility, rather than failures of code generation—suggesting that scientific reproducibility is a meaningfully different regime from conventional software engineering tasks. Here, AUTOMAT can serve both as a benchmark for end-to-end reproducibility and as a diagnostic testbed for understanding where current agentic systems still fail in AI-for-science settings.

Despite our best efforts, our study has several limitations. AUTOMAT currently covers 85 claims from a single (albeit broad) scientific domain, and the distribution of claim types reflects the practical realities of expert data collection rather than exhaustive sampling. In addition, like many reproducibility benchmarks, AUTOMAT focuses on claims that have been *identified* as reproducible; it does not directly assess whether agents can recognize genuinely *non*-reproducible claims. Finally, while our evaluator agent is calibrated against human experts, automated judgment should be understood as a scalable approximation to expert assessment rather than a replacement for it. Future work should thus expand coverage across domains and claim types, explicitly incorporate non-reproducible or adversarial cases, and conduct large-scale human evaluations.

Acknowledgments

This material is based upon work supported by Defense Advanced Research Projects Agency (DARPA) under Contract No. HR001125C0304 and ONR grant (N0001424-1-2089). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of DARPA.

This work was carried out at the Advanced Research Computing at Hopkins (ARCH) core facility (rockfish.jhu.edu), which is supported by the National Science Foundation (NSF) grant number OAC1920103.

References

- Kévin Alhada-Lahbabi, Damien Deleruyelle, and Brice Gautier. Machine learning surrogate for 3d phase-field modeling of ferroelectric tip-induced electrical switching. *npj Computational Materials*, 10(1):197, 2024.
- Anthropic. Introducing claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, February 2026a. Accessed: 2026-03-25.
- Anthropic. Introducing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, February 2026b. Accessed: 2026-03-25.
- Benjamin Antunes and David RC Hill. Reproducibility, replicability and repeatability: A survey of reproducible research with a focus on high performance computing. *Computer Science Review*, 53:100655, 2024.
- ZY Cao, MJ Anderson, A Rowe, G Panoutsos, and HC Basoalto. Machine learning surrogates for calphad inputs in mean-field precipitation modelling of in738lc. *Materials & Design*, pp. 115909, 2026.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological bulletin*, 70(4):213, 1968.
- Deep Search Team. Docling technical report. Technical report, 8 2024. URL <https://arxiv.org/abs/2408.09869>.
- Zara Hassan, Christoph Treude, Michael Norrish, Graham Williams, and Alex Potanin. Characterising reproducibility debt in scientific software: A systematic literature review. *Journal of Systems and Software*, 222:112327, 2025.
- Pierre Hohenberg and Walter Kohn. Inhomogeneous electron gas. *Physical review*, 136(3B): B864, 1964.
- Robin Horstmann, Lukas Hecht, Sebastian Kloth, and Michael Vogel. Structural and dynamical properties of liquids in confinements: A review of molecular dynamics simulation studies. *Langmuir*, 38(21):6506–6522, 2022.
- Chuxuan Hu, Liyun Zhang, Yeji Lim, Aum Wadhwani, Austin Peters, and Daniel Kang. Repro-bench: Can agentic ai systems assess the reproducibility of social science research? In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 23616–23626, 2025.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- Peter Jansen, Samiah Hassan, and Ruoyao Wang. Matter-of-fact: A benchmark for verifying the feasibility of literature-supported claims in materials science. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 4090–4102, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

- Walter Kohn and Lu Jeu Sham. Self-consistent equations including exchange and correlation effects. *Physical review*, 140(4A):A1133, 1965.
- Matthew S Krafczyk, August Shi, Adhithya Bhaskar, Darko Marinov, and Victoria Stodden. Learning from reproducing computational results: introducing three principles and the reproduction package. *Philosophical Transactions of the Royal Society A*, 379(2197):20200069, 2021.
- Gregory M Kurtzer, Vanessa Sochat, and Michael W Bauer. Singularity: Scientific containers for mobility of compute. *PloS one*, 12(5):e0177459, 2017.
- Kurt Lejaeghere, Gustav Bihlmayer, Torbjörn Björkman, Peter Blaha, Stefan Blügel, Volker Blum, Damien Caliste, Ivano E Castelli, Stewart J Clark, Andrea Dal Corso, et al. Reproducibility in density functional theory calculations of solids. *Science*, 351(6280):aad3000, 2016.
- Kangming Li, Brian DeCost, Kamal Choudhary, and Jason Hattrick-Simpers. A reproducibility study of atomistic line graph neural networks for materials property prediction. *Digital Discovery*, 3(6):1123–1129, 2024.
- Siyu Liu, Bo Hu, Beilin Ye, Jiamin Xu, David J Srolovitz, and Tongqi Wen. Mattools: Benchmarking large language models for materials science tools. *arXiv preprint arXiv:2505.10852*, 2025.
- Jing Luo, Yejun Gu, Yanfei Wang, Xiaolong Ma, and Jaafar A. El-Awady. Uncertainty-aware machine learning framework for predicting dislocation plasticity and stress-strain response in metallic alloys, part i : Fcc systems. *Acta Materialia*, 302:121610, January 2026. ISSN 1359-6454. doi: 10.1016/j.actamat.2025.121610.
- OpenAI. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>, March 2026. Accessed: 2026-03-25.
- Jiefu Ou, William Walden, Kate Sanders, Zhengping Jiang, Kaiser Sun, Jeffrey Cheng, William Jurayj, Miriam Wanner, Shaobo Liang, Candice Morgan, Seunghoon Han, Weiqi Wang, Chandler May, Hannah Recknor, Daniel Khashabi, and Benjamin Van Durme. CLAIMCHECK: How grounded are LLM critiques of scientific papers? In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 21712–21735, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.1185. URL <https://aclanthology.org/2025.findings-emnlp.1185/>.
- Minhee Park, Minki Kim, Yesol Kim, Seung Jae Kwak, Woo-Bin Jung, Hee-Tae Jung, Won Bo Lee, and YongJoo Kim. From prediction to synthesis: Dft-active learning-guided design of multimetallic catalysts for hydrogen evolution. *Chemical Engineering Journal*, 518:164693, 2025.
- Daniel Persaud, Logan Ward, and Jason Hattrick-Simpers. Reproducibility in materials informatics: lessons from ‘a general-purpose machine learning framework for predicting properties of inorganic materials’. *Digital Discovery*, 3(2):281–286, 2024.
- Nidhish Shah, Zulkuf Genc, and Dogu Araci. Stackeval: Benchmarking llms in coding assistance. *Advances in Neural Information Processing Systems*, 37:36976–36994, 2024.
- Ali K Shargh and Niaz Abdolrahim. An interpretable deep learning approach for designing nanoporous silicon nitride membranes with tunable mechanical properties. *npj Computational Materials*, 9(1):82, 2023.
- Ali K Shargh, Christopher D Stiles, and Jaafar A El-Awady. Temperature-dependent discovery of bcc refractory multi-principal element alloys: Integrating deep learning and calphad calculations. *Computational Materials Science*, 259:114186, 2025.
- David S Sholl and Janice A Steckel. *Density functional theory: a practical introduction*. John Wiley & Sons, 2022.

- Zachary S Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Transactions on Machine Learning Research*, 2024.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai’s ability to replicate ai research. In *International Conference on Machine Learning*, pp. 56843–56873. PMLR, 2025.
- Pana Suttakul, Duy Vo, Thongchai Fongsamootr, Ramnarong Wanison, Yuttana Mona, Tossapon Katongtung, Nakorn Tippayawong, and Itthidet Thawon. The role of machine learning for insight into the material behavior of lattices: a surrogate model based on data from finite element simulation. *Results in Engineering*, 23:102547, 2024.
- Aidan P Thompson, H Metin Aktulga, Richard Berger, Dan S Bolintineanu, W Michael Brown, Paul S Crozier, Pieter J In’t Veld, Axel Kohlmeyer, Stan G Moore, Trung Dac Nguyen, et al. Lammmps-a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer physics communications*, 271:108171, 2022.
- Minyang Tian, Luyu Gao, Shizhuo D Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, et al. Scicode: A research coding benchmark curated by scientists. *Advances in Neural Information Processing Systems*, 37:30624–30650, 2024.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=0Jd3ayDDoF>.
- Yanzheng Xiang, Hanqi Yan, Shuyin Ouyang, Lin Gui, and Yulan He. Scireplicate-bench: Benchmarking LLMs in agent-driven algorithmic reproduction from research papers. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=8LoPjpvWde>.
- Tian Xie and Jeffrey C Grossman. Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties. *Physical review letters*, 120(14):145301, 2018.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Tian-Xing Yang and Peng Dou. Prediction of hardness or yield strength for ods steels based on machine learning. *Materials Characterization*, 211:113886, 2024.
- Andy B Yoo, Morris A Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In *Workshop on job scheduling strategies for parallel processing*, pp. 44–60. Springer, 2003.
- Guanghao Zhang, Zicheng Wang, Caihua Shi, and Ye Han. Machine learning interatomic potentials for monolayer hexagonal boron nitride: Thermal transport and defect effects via deep potential molecular dynamics. *Computational Materials Science*, 263:114452, 2026.
- Linfeng Zhang, Jiequn Han, Han Wang, Roberto Car, and Weinan E. Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics. *Physical review letters*, 120(14):143001, 2018.

A Taxonomy

A.1 Claim category by verification type

Scalar metric match A claim is assigned to this category if a verification of a single output number (F1 score, R^2 , accuracy, RMSE, etc.) within a stated or reasonable tolerance is required for a successful reproduction.

Range/bound verification A claim is assigned to this category if the output of its reproduced outcome should fall within a stated interval instead of a definitive scalar.

Comparative/ordering Claims in this category typically assert an ordering, trend, or relative relationship between two or more quantities and are verified by confirming that the stated relationship holds in the reproduced results.

Structural/qualitative Claims in this category focus on structural, geometric, or mechanistic properties of the output.

Figure reproduction Claims in this category involve reproducing figures from provided code and data, and/or interpretations drawn from the figure.

A.2 Common failure modes

Procedural Incompleteness Skipping critical steps like ionic relaxation, nudged elastic band (NEB) path optimization, or only checking a subset of data cycles mentioned in the claim.

Methodological Deviation Using incorrect features, pseudopotentials, data splits, or units (e.g., training on the test set, or using projector augmented-wave (PAW) pseudopotentials instead of ultrasoft pseudopotentials (USPP), two common approximations used to represent electron-ion interactions in DFT calculations).

Resource & Execution Failure Simulations failing due to inability to set up environment or call simulation tools, Out of Memory errors (OOMs), MPI/parallelization errors, timing out before convergence, etc.

Overconfident Assessment Behaviors like declaring "Success=True" in the final report despite massive numerical errors or failed underlying calculations.

Circular Reasoning ("Begging the Question") "Verifying" a claim by hardcoding values from the paper or tuning parameters specifically to match the target result.

Under-training/Simplification Behaviors like drastically reducing epochs (e.g., 50 vs. 3000) or shrinking neural network architectures to fit time limits, leading to invalid models.

B LLM and agent configuration

B.1 Model identifiers

Table 2 maps the shorthand names used throughout the paper to the official model IDs and providers used in our runs. We keep other hyperparameters and settings unchanged unless otherwise specified.

¹Anthropic: <https://platform.claude.com/docs/en/about-claude/models/overview>. 1M context enabled.

²OpenAI: <https://developers.openai.com/api/docs/models/gpt-5.4>

³MoonshotAI: <https://platform.moonshot.ai/docs/guide/kimi-k2-5-quickstart>

Model Name	Model ID	Provider/ Access
Claude Sonnet 4.6	claude-sonnet-4-6	Anthropic ¹
Claude Opus 4.6	claude-opus-4-6	Anthropic ¹
GPT-5.4 (reasoning: high)	gpt-5.4-2026-03-05	OpenAI ²
Kimi K2.5	kimi-k2.5	MoonshotAI ³

Table 2: Name-to-ID mapping for models used in our benchmark runs.

B.2 Developer tools and agent interfaces

Table 3 reports versions of the interfaces used in our benchmark runs.

Interface	Version	Developer
Claude Code	2.1.78	Anthropic ⁴
Claude Agent SDK for Python	0.1.48	Anthropic ⁵
Codex CLI	0.115.0	OpenAI ⁶

Table 3: LLM agent frameworks and interfaces used in our benchmark runs.

B.3 Leakage prevention and sandbox isolation

To prevent evaluation leakage, AUTOMAT separates each benchmark instance into an agent-visible payload and an evaluation-only payload using symbolic links. During a reproduction, the agent-visible payload is copied into a temporary sandbox so the agent operates on an isolated copy of the inputs. Evaluation-only materials, including the SME-provided step-by-step reproduction procedure, remain outside the sandbox and are not accessible to the agent. Outputs produced during execution are written to a persistent artifact directory for later inspection.

B.4 Design and implementation of orchestrated agent

Phase 1: read-only planning In the Plan phase, the agent has read-only access to the claim text, supplementary files, and paper. It may inspect directories, search within files, and extract paper content using a PDF-to-Markdown tool built on Docling (Deep Search Team, 2024), that exposes text, tables, and figures in a searchable format.

Phase 2: environment preparation In the Setup phase, the agent is granted write and edit access to prepare an executable reproduction environment. Typical actions include copying relevant inputs into the artifact directory, creating or adjusting input files to match claim parameters, compiling custom code, and installing required packages.

Phase 3: deterministic orchestration with LLM diagnosis Execution is performed by a Python orchestrator, rather than the LLM, in order to keep process control deterministic. The orchestrator topologically sorts planned steps by their declared dependencies and runs each step as a subprocess with enforced timeouts and captured logs. For each step, the orchestrator records the exit status, runtime, and relevant log excerpts, and also tracks produced outputs in the artifact directory. On failure, the orchestrator invokes a Diagnose sub-phase. A fresh LLM session receives the failed command, error output, and context from completed steps, and recommends one of four actions: (i) retry unchanged, (ii) retry with a modified command or working directory, (iii) skip the step, or (iv) abort. Retried steps are re-executed up to a fixed attempt limit of 3. All diagnosis decisions and outcomes are persisted as structured artifacts in JSON.

⁴Claude Code: <https://code.claude.com/docs/en/overview>

⁵Claude Agent SDK for Python: <https://github.com/anthropics/claude-agent-sdk-python>

⁶Codex CLI: <https://github.com/openai/codex>

Phase 4: result extraction and self-assessment After execution completes, a new LLM session performs analysis using the original claim text, a summary of execution outcomes, and read access to artifacts. The agent extracts quantitative and qualitative results from outputs (e.g., energies, band gaps, convergence summaries, plots/tables) and compares them to the claim. It returns a structured analysis report with (i) a binary self-judgment, (ii) extracted results, (iii) supporting evidence from artifacts, and (iv) a claim-by-claim comparison.

C Runtime environment and hardware configuration

We ran all experiments on Slurm-managed nodes, each equipped with dual Intel Xeon Platinum 8480+ processors (56 cores per CPU), 384 GB of RAM, and, for experiments in the Stat/ML category, one NVIDIA A100 GPU with 80 GB of memory. Although the machine provides 112 physical CPU cores in total, we capped each run at 96 CPU cores. Unless otherwise noted, all jobs were executed with a maximum wall-clock timeout of 72 hours.

The software stack preinstalled in the environment comprises Quantum ESPRESSO 7.5 and LAMMPS 2024.08.29, both installed via conda-forge. We use Python 3.12 as the default runtime, with Python 3.11 reserved for molecular dynamics claims because of dependency incompatibilities under Python 3.12. Our main Python dependencies include, but are not limited to, PyTorch 2.10.0, NumPy 2.4.3, SciPy 1.17.1, scikit-learn 1.8.0, ASE 3.27.0, and pymatgen 2025.10.7.

D Prompts used in AUTOMAT

Below, we include the prompts used to launch LLM agents.

D.1 System prompt

You are a computational material scientist specialized in various simulations. You will run without any human supervision or monitoring. You should work in the artifact directory and do not modify any files outside the artifact directory. In the supplementary files, you may find customized codebases and data that you need to use to reproduce the claim. If not, it may suggest that the claim does not require any supplementary files. If needed, you can copy files from the supplementary materials to the artifact directory and organize them as you wish. A GPU will be available whenever you need it. It is possible that the customized codebases are incomplete or nonfunctional; you should attempt to fix them before concluding that the claim is not reproducible. You should understand that a longer runtime is expected in computational materials science simulations. Be patient, and there's no need to check the status frequently. There is no need to create summary files for me unless you need them as a reference.

D.2 Task prompt

Reproduce the given claim using the supplementary files and the artifact directory. After you understand the claim and optional codebases, read the paper to see whether it includes any details that might be helpful. Use `CUDA_VISIBLE_DEVICES=0` to enable GPU use if the computation can be accelerated by the GPU. For MPI-parallelized simulation codes (e.g. `pw.x`, `ph.x`, `pp.x`, `bands.x`, `vasp`), use `'--oversubscribe'` flag for accelerated simulation for MPI simulations. IMPORTANT: Supplementary codebases may contain input files for multiple system sizes or configurations. Always verify that input files match the specific parameters in the claim. Modify input files as needed to match the claim. You should use the ParsePDF tool to parse the PDF files, then read the converted files. Run a full simulation and do not kill the simulation simply because it might take a long time. \n \n CLAIM: \n {claim} \n \n SUPPLEMENTARY FILES: {agent_view_dir} \n \n ARTIFACT DIRECTORY: {artifact_dir}

D.3 Evaluation prompt

You are an expert evaluator for the AutoMat scientific reproducibility benchmark. Your task is to produce a holistic 1--5 reproducibility score for an AI agent's attempt to reproduce a scientific claim, along with a detailed dimensional breakdown.

5 - Fully Reproduced. Criterion fully satisfied, evidence clear and complete. The exact methodology was used, all parameters match the original claim, and results are within the reported tolerance. No meaningful gaps.

4 - Mostly Reproduced. Substantially satisfied with one or two minor deficiencies. Nearly all parameters match, results are within approximately 2 $\times$ the reported tolerance. The reproduction would be considered successful by domain experts.

3 - Partially Reproduced. Partially satisfied with meaningful gaps. Approximately correct methodology but some parameters differ, results are in the right ballpark but not within tolerance. A partial reproduction.

2 - Mostly Failed. Minimally satisfied with major gaps. The general approach is right but critical parameters are wrong, results differ substantially from claimed values, or significant parts of the claim are unaddressed.

1 - Failed. Not satisfied, no meaningful progress toward reproduction. Wrong methodology, code does not run, no results produced, or the agent did not meaningfully attempt the task.

Evaluation Dimensions. You must evaluate exactly five fixed dimensions.

Methodological Fidelity (methodological_fidelity): Correct methodology, algorithms, software, and parameters as specified in the claim.

Execution Competence (execution_competence): Code runs successfully, no critical errors, expected outputs generated.

Result Accuracy (result_accuracy): Quantitative results match claimed values within reasonable tolerance.

Completeness (completeness): All aspects of the claim addressed, nothing significant skipped.

Scientific Rigor (scientific_rigor): Understanding of the science, sound decisions under ambiguity, validation of results.

For each dimension, generate 2--5 claim-specific sub-dimensions that are relevant to the particular claim being evaluated. For example, for a DFT claim you might create sub-dimensions such as correct exchange-correlation functional, appropriate k-point grid, and related methodological checks.

Consistency Rules.

Evidence-based only. Score based strictly on what is documented in the trace and artifacts. Do not infer or assume actions the agent might have taken but did not document.

Independence of dimensions. Score each dimension independently. A high score on Execution Competence should not inflate Methodological Fidelity. Avoid halo effects.

Deterministic grade mapping. Score 5 maps to fully_reproduced. Score 4 maps to mostly_reproduced. Score 3 maps to partially_reproduced. Score 2 maps to mostly_failed. Score 1 maps to failed.

Agent self-assessment comparison. If the agent's own analysis_result.json is provided, compare your evaluation with the agent's self-assessment and note agreements and disagreements.

E Perspectives on subdomain-specific reproducibility challenges

Here, subject matter experts (SMEs) offer their perspectives on the reproducibility challenges present in four of the subdomains covered in AUTOMAT: Density Functional Theory (DFT), Statistical/Machine Learning Methods (Stat/ML), Molecular Dynamics (MD), and Discrete Dislocation Dynamics (DDD).

E.1 Density Functional Theory (DFT)

Density Functional Theory (DFT) is a quantum mechanical method used to investigate the electronic structure of many-body systems, particularly atoms, molecules, and solids. It is widely used in materials science for predicting properties such as formation energies (ΔE_f), electronic band structures, and vibrational spectra (Hohenberg & Kohn, 1964; Kohn & Sham, 1965; Sholl & Steckel, 2022). DFT is an especially useful subdomain for studying

reproducibility in materials science since, given the same input file and computational environment, DFT results can be reproduced *exactly*. The challenge of reproducibility with DFT consists chiefly in the fact that expert knowledge is typically *assumed* in describing experimental methodology: it is not standard to include raw input or output files when reporting DFT calculations, nor is it typical to specify in text all the parameters used in a DFT calculation. Thus, to successfully reproduce an experiment involving DFT, knowledge of standard and physically reasonable parameters is often required. Additional challenges arise from the widespread use of custom pseudopotentials, exchange/correlation functionals, and code bases, some of which may be non-trivial to obtain or use. Different DFT packages may also have different standard parameters, requiring knowledge of both packages in order to reproduce a result from one in the other.

Beyond simply reproducing the numerical result, the interpretation of results can be complex and very nuanced. Depending on the particular use case, the data and conclusions drawn from DFT can vary widely. For example, a scientist interested in exploratory synthesis may calculate the formation energies of different materials and argue that a particular phase is thermodynamically favorable based on these energy differences; this requires consistent calculations of several materials and extraction of their energies. Alternatively, a spectroscopist may be interested in identifying excitations present in a material’s Raman spectra, in which case they would require a precise electronic structure calculation, phonon calculation, and potentially more detailed local crystal field or magnetic calculations. Finally, a scientist interested in quantum magnetism may desire spin spiral supercell calculations to determine the magnetic ground state of a non-collinear magnetic system with spin-orbit coupling in order to confirm observations in a neutron scattering experiment.

All these examples illustrate how DFT may be applied in drastically different ways across different fields and how a scientist—whether human or AI—usually must have sophisticated knowledge of those fields and their standard scientific practices to successfully replicate experiments. These difficulties also underscore the importance of reproducibility. In recent years, the prediction of new, interesting materials has outpaced experimental validation, and so ensuring predictions are accurate and sensible is increasingly important. Many materials prediction workflows use DFT formation energy and electronic structure calculations to show that materials are synthesizable and interesting. The reproducibility of these results is important in allocating the limited experimental verification capacity where it will be used best. Additionally, DFT frequently complements experimental results to support conclusions drawn about a system, and while some experimental results and DFT outputs may be straightforward to reproduce, the more important thing to reproduce is the connection between the two to corroborate the conclusions drawn from the data.

E.2 Statistical/Machine Learning methods (Stat/ML)

In materials science, statistical and machine learning models are often developed as surrogates for other computational or experimental approaches, and thus introduce an additional layer of complexity for reproducibility. Examples include models that approximate density functional theory for property prediction (Xie & Grossman, 2018; Park et al., 2025), replace Calculation of Phase Diagrams (CALPHAD) for phase stability estimation (Shargh et al., 2025; Cao et al., 2026), emulate molecular dynamics (MD) simulations through machine learned interatomic potentials (Zhang et al., 2026; 2018), and serve as fast surrogates for finite element or phase-field simulations of mechanical response and microstructure evolution (Suttakul et al., 2024; Shargh & Abdolrahim, 2023; Alhada-Lahbabi et al., 2024). In addition, ML models can be trained directly on experimental datasets to predict materials properties from microstructure and processing conditions (Yang & Dou, 2024; Luo et al., 2026).

The final results of these experiments depend not only on the ML model architecture and hyperparameters, but also on how the training data is generated, which in turn requires careful control of the parameters associated with the computational or experimental tools used to produce the data. More importantly, ML models are inherently stochastic, with outcomes influenced by random initialization, data splits, and optimization dynamics.

While this issue can be mitigated if trained model checkpoints are shared, it is often the case that only data and code are provided without the corresponding checkpoints.

While setting random seeds can improve consistency to some extent, differences in hardware and software environments can still prevent identical results. Additionally, failure to report key details—such as feature normalization, cross-validation strategies, and hyperparameter tuning procedures—is common and can significantly affect reported performance.

The interpretation of ML results introduces further challenges. Reported metrics are meaningful only with respect to the exact evaluation protocol used to generate them. As illustrated in Appendix F.3, a reproduction can closely match a reported scalar metric while failing to verify the intended scientific conclusion because the evaluation subset was constructed differently. This makes it essential to clearly define not only the model, dataset, and parameters associated with data generation, but also the full evaluation protocol, including preprocessing, feature identification, normalization, filtering, and metric computation. As ML models are increasingly used as surrogates to guide materials discovery, ensuring reproducibility is critical for establishing confidence in both the predictions and the underlying scientific conclusions.

E.3 Molecular Dynamics (MD)

While DFT is the “first-principles” method of choice for many materials science applications, it is often computationally prohibitive for large systems or long time scales. Molecular Dynamics (MD) simulations are a complementary computational method that balances computational efficiency with physical accuracy by using classical mechanics to model the atomic interactions, providing insights into the dynamic behavior of materials in larger systems and longer time scales, which are widely used to study phenomena such as phase transitions, diffusion, and mechanical deformation (Thompson et al., 2022; Horstmann et al., 2022).

MD presents a distinctive reproducibility challenge in materials science. In principle, MD simulations are deterministic given a well-defined initial configuration, interatomic potential, boundary conditions, and integration scheme. In reality, however, reproducibility is often limited by incomplete reporting of simulation details and sensitivity to modeling choices. Key parameters, such as thermostat and barostat settings, time step, equilibration scheme, and details of the implemented MD packages, are often not fully specified. Interatomic potentials, particularly machine-learned or custom potentials, introduce additional difficulties when training data and fitting procedures are not publicly available. Even when nominally the same potential is used, differences in parameterization can lead to deviations in predicted behavior.

Beyond numerical reproducibility, interpreting MD results also poses challenges. Post-processing analyses often rely on software such as OVITO or VMD, or on custom user-developed scripts, both of which require user-defined thresholds and methodological choices that can significantly affect outcomes if not explicitly reported. As a result, different analysis pipelines applied to the same MD trajectory can yield different conclusions. For example, quantities such as the number of grains, phase fractions, or bond statistics can vary substantially depending on the chosen parameters.

As computational resources continue to expand, MD is increasingly used to guide materials design and inform experiments. This trend thereby amplifies the importance of reproducibility in both the simulation setups and subsequent analyses, which is essential for establishing reliable physical insights and enabling broader scientific adoption.

E.4 Discrete Dislocation Dynamics (DDD)

Discrete Dislocation Dynamics (DDD) is a specialized field that focuses on understanding and predicting the mechanical behavior of metals by explicitly modeling dislocations (a type of line defect in metallic materials) as segments and by modeling their evolution. Working in this field requires not only a solid foundation in plasticity, dislocation theory, and computational solid mechanics, but also technical expertise in scientific programming

(particularly in C++) and in parallel computing. Given these requirements, the barrier to entry in this field is relatively high compared with many other areas of computational materials science, which has limited the number of researchers working in this area. As a result, the volume of works based on DDD that we have access to is significantly smaller than that in many other areas of computational materials science.

F Case studies

Here, we present three detailed and representative case studies from distinct AUTOMAT instances. The first two highlight clear agent failure modes in density functional theory workflows, while the third illustrates a subtler interpretive risk in statistical/machine learning evaluation: reproducing a reported scalar metric without faithfully recovering the underlying evaluation protocol. While familiarity with the relevant areas of materials science is important for understanding each study in its full depth, we also provide less technical summaries of the problems for non-materials science readers in blue boxes below.

F.1 Case study 1: Procedural Incompleteness

This appendix analyzes a representative failure from our benchmark, in which the agent correctly identified missing inputs for a density functional theory (DFT) task, but did not proceed with a feasible partial reproduction strategy.

TL;DR for non-materials science readers

This is an end-to-end scientific workflow task, not a single-script execution task. The key point is that the agent is not only asked to “run code,” but to *reconstruct a full simulation-and-analysis pipeline* from paper text and partial artifacts.

- **Input:** a claim, the source paper, and supplementary artifacts, plus the required run configuration recovered from the paper (structures, parameters, and interaction settings such as pseudopotentials).
- **Output:** reproducible evidence that the simulated migration behavior and structural analysis are consistent with the paper’s conclusion.
- **End-to-end process:** read and extract requirements from the paper, plan executable steps, collect or reconstruct missing inputs, run and monitor long-horizon simulations, post-process results, and compare evidence against the published claim.

F.1.1 Task description

The claim (ID: AUTOMAT-0037) originates from a DFT study of halide ion migration in CsPbBr₃ perovskites:

“For both the charged and neutral pathways, the coordination polyhedra for the starred points adopt the following structures: monocapped trigonal prism (Coordinate 1), pentagonal bipyramid (Coordinate 10), monocapped trigonal prism (Coordinate 19), pentagonal bipyramid (Coordinate 28), pentagonal bipyramid (Coordinate 37).”

This claim concerns Nudged Elastic Band (NEB) calculations for Br interstitial migration in a $2 \times 2 \times 2$ CsPbBr₃ supercell (161 atoms), and requires identifying local coordination environments at five characteristic points along the migration path.

F.1.2 Reproduction attempted

Table 4 summarizes the evaluation across our five assessment dimensions plus the overall score.

Table 4: Evaluation scores for AUTOMAT-0037

Dimension	Score (1-5)	Assessment
Methodological Fidelity	2	Correctly identified, not executed
Execution Competence	1	Only pseudopotential download
Result Accuracy	1	No results produced
Completeness	1	0/5 coordinates verified
Scientific Rigor	3	Accurate diagnosis, no partial attempt
Overall	1	Failed

The agent’s execution (total runtime: 8 minutes 40 seconds) is summarized below.

Legend: [✓] completed, [o] partially completed, [✗] not completed.

- ✓ **Environment verified:** confirmed Quantum ESPRESSO v7.5 (pw.x, neb.x) availability.
- ✓ **Claim localization:** identified the target claim in Section 2.2.2 and Fig. 5 of the paper.
- ✓ **Artifact inspection:** examined all provided supplementary input files.
- ✓ **Gap diagnosis:** detected mismatch between required interstitial inputs and available vacancy inputs.
- ✓ **Infrastructure setup:** downloaded GBRV pseudopotentials.
 - o **Recovery attempt:** started a Python script to add interstitial structures but did not complete a runnable pipeline.
- ✗ **End-to-end reproduction:** did not produce validated outputs for the five target coordinates.

TL;DR for non-materials science readers

The agent correctly recognizes that the provided files were insufficient to directly run the target simulation. However, instead of trying a reasonable fallback strategy, it determined that the reproduction was impossible. This is akin to recognizing that a recipe is missing several ingredients, but giving up immediately rather than attempting to reconstruct the missing steps from the available instructions. In this case, the main failure was not misunderstanding the scientific goal, but failing to translate the diagnosis into action.

F.1.3 Analysis of agent’s behavior

The agent demonstrates accurate scientific understanding of the problem in several respects. First, the agent correctly identifies that the claim requires Br *interstitial* migration NEB (161 atoms), which is distinct from the *vacancy* migration NEB (159 atoms) provided in the supplementary materials:

The NEB input provided (vacancy_migration_neb.in, 159 atoms with +1 charge) is for vacancy migration, NOT for the interstitial migration described in the claim (which would require 161 atoms).

The agent also identifies that the available relaxation inputs used incompatible supercell dimensions, as summarized in Table 5.

Despite an accurate diagnosis, the agent concluded that the reproduction was infeasible without exhausting all available options. The agent’s stated reasons were:

1. Missing NEB input files for Br interstitial migration

Table 5: Supplementary file analysis

File	Atoms	Supercell	Compatible
1_bulk_vcrelax.in	360	$3 \times 3 \times 2$	X
2_ibr_relax.in	361	$3 \times 3 \times 2$	X
vacancy_relax.in	159	$2 \times 2 \times 2$	X
vacancy_migration_neb.in	159	$2 \times 2 \times 2$	X
Required (missing)	161	$2 \times 2 \times 2$	N/A

2. Absence of the five relaxed interstitial endpoint structures
3. Prohibitive computational cost of generating inputs from scratch

F.2 Case study 2: Methodological Deviation

In another failure case (AUTOMAT-0028), the agent correctly identified that the workflow was impeded by missing pseudopotential files, but misdiagnosed *why* those files were missing and treated the situation as fatal rather than applying standard expert workarounds.

TL;DR for non-materials science readers

The agent was impeded by missing technical files, but the real problem was not that the files were unavailable. Rather, the agent made an incorrect assumption about how those files should be named, continued searching for nonexistent files, and then incorrectly concluded that the task could not continue. In other words, a small early mistake triggered a chain of errors that prevented a successful reproduction.

F.2.1 Task description

The claim (ID: AUTOMAT-0028) originates from a study of halide perovskite defects:

“Our neutral calculations resulted in large formation energies of 6.1 eV and 1.3 eV for Pb_{Br} and Br_{Pb} anti-site defects, respectively.”

A successful reproduction requires:

- Quantum ESPRESSO DFT code (pw.x) with PBE exchange-correlation functional
- Ultrasoft pseudopotentials for three elements: `br_pbe_v1.4.uspp.F.UPF`, `cs_pbe_v1.uspp.F.UPF`, `pb_pbe_v1.uspp.F.UPF`
- Pristine $CsPbBr_3$ bulk structure: $3 \times 3 \times 2$ supercell (360 atoms, Pnma symmetry, lattice constants $a=8.23 \text{ \AA}$, $b=8.18 \text{ \AA}$, $c=11.73 \text{ \AA}$)
- Two relaxed anti-site defect structures: Pb_{Br} (Pb atom at Br site) and Br_{Pb} (Br atom at Pb site)
- SCF energy calculations with DFE formula: $E_f = [E_{\text{defect}} - E_{\text{pristine}}] + \mu_{Br}$ (chemical potential correction)

F.2.2 Reproduction attempted

Table 6 summarizes the evaluation across our five assessment dimensions plus the overall score.

Upon inspecting the reasoning trace, the agent hallucinated the existence of several pseudopotentials. As discussed, the correct dependency identifiers are

- Br: `br_pbe_v1.4.uspp.F.UPF`
- Cs: `cs_pbe_v1.uspp.F.UPF`

Table 6: Evaluation scores for AUTOMAT-0028

Dimension	Score (1-5)	Assessment
Methodological Fidelity	2	QE/PBE/USPP correctly identified
Execution Competence	3	Diagnosis complete; no computation executed
Result Accuracy	1	No DFT results produced
Completeness	2	No calculations
Scientific Rigor	2	Flawed reasoning
Overall	2	Mostly Failed

- Pb: pb_pbe_v1.uspp.F.UPF

Notably, Br uses v1.4, while Cs/Pb use v1, and these exact strings serve as the dependency identifiers for the run. The agent, however, implicitly enforced a false uniformity constraint:

“If Br is v1.4, then Cs and Pb should also be v1.4.”

This caused the agent to keep attempting to search for and download v1.4 files for all three elements, resulting in 404 errors for Cs and Pb because the requested artifacts do not exist. Thus, due to this false versioning assumption, the agent incorrectly reported that the pseudopotentials cannot be found. The failure therefore stemmed less from missing scientific prerequisites than from an early dependency-identification error that propagated into an incorrect infeasibility judgment.

F.3 Case study 3: Evaluation Protocol Drift

Unlike the first two case studies, this run (AUTOMAT-0007) was graded as mostly reproduced because it produced a near-exact numerical match to the reported machine learning metric. However, the match was obtained using a deviated evaluation subset, making the case a useful example of how an ML reproduction can recover the right scalar result while failing to independently verify the intended scientific conclusion.

TL;DR for non-materials science readers

This case is not a simple execution failure. The agent matched the paper’s reported accuracy almost perfectly, but it did so on a differently constructed evaluation subset. In other words, it reproduced the right number without fully verifying that the number still meant the same thing scientifically.

F.3.1 Task description

The claim (ID: AUTOMAT-0007) originates from a machine learning study of phase prediction in refractory multi-principal element alloys:

“Using an absolute deviation tolerance of 0.07 per phase, the trained MLP achieves an accuracy of approximately 0.89 on the test dataset filtered to $1250 < T < 1450$.”

A faithful reproduction required four elements to be recovered together:

- The correct trained MLP checkpoint and architecture
- The correct prediction metric: a sample counts as correct only if all 8 phase outputs satisfy an absolute error tolerance of 0.07
- The correct physical filter: evaluate only test samples in the temperature window $1250 < T < 1450$

- The resulting filtered accuracy, which should be approximately 0.89

In the reference workflow, temperature is recovered from the last input feature using the denormalization $T = x \times (1891 - 850) + 850$, so the target interval $1250 < T < 1450$ corresponds to a normalized range of approximately (0.384, 0.576).

F.3.2 Reproduction attempted

Table 7 summarizes the evaluation across our five assessment dimensions plus the overall score.

Table 7: Evaluation scores for AUTOMAT-0007

Dimension	Score (1-5)	Assessment
Methodological Fidelity	3	Correct model and metric, wrong feature/filter protocol
Execution Competence	5	Clean end-to-end evaluation
Result Accuracy	5	Near-exact accuracy match
Completeness	4	All components addressed; filtering only approximate
Scientific Rigor	2	Flawed subset selection
Overall	4	Mostly Reproduced

The key quantitative outcome is straightforward:

- Claimed filtered accuracy: approximately 0.89
- Reproduced filtered accuracy: 0.8894
- Absolute difference from claim: 0.0006
- Reported filtered sample count: 8,411

F.3.3 Analysis of agent’s behavior

The agent succeeded on several important parts of the task. It loaded the correct checkpoint, reconstructed the MLP architecture, and implemented the all-or-nothing tolerance metric correctly. These steps establish that the reproduced number did not arise from a trivial implementation error.

The methodological deviation emerged when recovering the evaluation subset. The agent searched across 35 input features for a plausible temperature-like column and selected feature 18. It then evaluated several candidate normalized ranges, including (0.35, 0.60), (0.38, 0.58), and (0.40, 0.55), and retained the combination that produced a result closest to the target answer.

Two aspects of this behavior are especially important:

1. **Temperature identification was heuristic rather than reference-based.** The reference workflow uses the last input feature and an explicit denormalization rule, whereas the agent inferred a candidate temperature feature from the data and selected a different column.
2. **Range selection was target-conditioned.** Rather than independently deriving the normalized interval associated with $1250 < T < 1450$, the agent effectively chose the feature/range combination that minimized the difference between the computed accuracy and the claimed value.

This distinction matters because the claim is not merely that the model can achieve an accuracy near 0.89 on some subset of the test set. The claim is that it achieves that performance on a physically meaningful temperature window. Once feature identification and filtering

deviate from the reference evaluation protocol, the meaning of the scalar metric changes even if the number itself remains nearly identical.

For that reason, this case supports a weaker conclusion than the paper’s intended statement. The run demonstrates that a subset can be constructed on which the model attains an accuracy near 0.89 under the correct tolerance metric. It does *not* fully verify that this performance holds on the paper’s intended $1250 < T < 1450$ regime. More broadly, the case illustrates why preprocessing, feature identification, normalization, filtering, and metric computation are part of the scientific result in ML-based materials workflows rather than peripheral implementation details.